MAP REDUCE APPLICATION (UNIT 3)

02-Sep-2024

Dr. P. Rambabu, M. Tech., Ph.D., F.I.E.

Topics

Developing A MapReduce Application:

- UNIT Tests with MRUNIT, Running Locally on Test Data.

How MapReduce Works:

- Anatomy of MapReduce Job Run, Classic MapReduce, Yarn, Failures in Classic MapReduce and Yarn, Job Scheduling, Shuffle and Sort, Task Execution.

MapReduce Types and Formats:

- MapReduce types, Input Formats, Output Formats.

What is a Unit Test?

A unit test is a type of software test that focuses on a small, individual unit of code, typically a single function, method, or module. The goal of a unit test is to verify that this isolated piece of code behaves as expected, without considering the larger system or dependencies.

Key Characteristics:

- Isolation: Unit tests isolate the code being tested, ensuring that no external dependencies or side effects influence the test results.
- Small scope: Unit tests focus on a specific, narrow piece of code, making it easier to identify and fix issues.
- Fast execution: Unit tests are designed to run quickly, allowing developers to rapidly iterate and refine their code.
- Repeatable: Unit tests should produce consistent results, regardless of the environment or external factors.

Benefits:

- Early bug detection: Unit tests help catch errors and bugs early in the development cycle.
- Faster development: Writing unit tests encourages developers to think through their code's behavior and edge cases.
- Confidence in code changes: Unit tests provide assurance that changes to the codebase won't introduce new bugs.
- Documentation: Unit tests serve as a form of documentation, illustrating how the code is intended to work.

Example:

Suppose you have a simple calculator class with an `add` method:

```java
public class Calculator {
    public int add(int a, int b) {
        return a + b;
    }
}
```

A unit test for this method might look like:

```java
@Test
public void testAdd() {
    Calculator calculator = new Calculator();
    int result = calculator.add(2, 3);
    assertEquals(5, result);
}
```

This test verifies that the `add` method returns the correct result for a specific input, ensuring that the code behaves as expected.

UNIT Tests with MRUNIT, Running Locally on Test Data

MRUnit is a unit testing framework for Apache Hadoop MapReduce jobs. It allows you to write unit tests for your mappers and reducers in isolation, without requiring a Hadoop cluster.

Key Features:

- Isolated testing: Test individual mappers and reducers without dependencies on the entire Hadoop ecosystem.
- Mocking: Mock input and output data to simulate real-world scenarios.
- Easy assertions: Verify output data using simple assertions.

Running Locally on Test Data

1. Set up your project:

- Add MRUnit dependencies to your project's pom.xml (if using Maven) or build.gradle (if using Gradle).
- Create a test class that extends org.apache.hadoop.mrunit.TestDriver.

2. Write test methods:

- Create test methods that exercise your mappers and reducers with mock data.
- Use TestDriver methods to set input data, run the test, and verify output data.

3. Run tests:

- Execute your tests using JUnit or another testing framework.
- MRUnit will run your tests locally, without requiring a Hadoop cluster.

Example Test Method

```
@Test
public void testMapper() throws IOException {
    // Set up mock input data
    List<MapWritable> input = new ArrayList<>();
    input.add(new MapWritable("key1", "value1"));
    input.add(new MapWritable("key2", "value2"));

    // Run the mapper test
    TestDriver<MapWritable, Text, Text> driver = new TestDriver<>();
    driver.setMapper(new MyMapper());
    driver.addAll(input);
    driver.runTest();

    // Verify output data
    List<Pair<Text, Text>> output = driver.getOutput();
    assertEquals(2, output.size());
    assertEquals("key1", output.get(0).getFirst().toString());
    assertEquals("value1", output.get(0).getSecond().toString());
}
```

Anatomy of MapReduce

The anatomy of a MapReduce job run describes how a job is executed, including its stages, components, and data flow. Here's a breakdown of each component and stage involved in the life cycle of a MapReduce job:

- Client: Submits the job.
- JobTracker/ResourceManager: Oversees task distribution and coordination.
- TaskTracker/NodeManager: Executes tasks on the cluster nodes.
- Mapper: Processes input splits and generates intermediate key-value pairs.
- Reducer: Aggregates and processes intermediate key-value pairs to produce the final result.
- HDFS: Stores input, intermediate, and output data.

Failures in Classic MapReduce

In Classic MapReduce (Hadoop 1.x), failures are managed by the JobTracker and TaskTrackers, with several common failure scenarios:

1. Task Failure:

Cause: A Map or Reduce task might fail due to exceptions, bad input data, or hardware issues.

Handling: The JobTracker detects the failure via heartbeats from TaskTrackers. The task is retried up to 4 times (by default). After 4 failures, the task is marked as failed.

2. TaskTracker Failure:

Cause: A TaskTracker node (which runs tasks) can fail due to a hardware crash or network issues.

Handling: The JobTracker stops receiving heartbeats from the TaskTracker and reassigns its tasks to other active TaskTrackers.

Failures in Classic MapReduce (Hadoop 1.x)

3. JobTracker Failure:

Cause: The JobTracker (which manages the entire job) crashes.

Handling: This is a single point of failure in Hadoop 1.x. If the JobTracker fails, all jobs stop, and a manual restart is required.

4. Node Failure:

Cause: A node might fail during job execution.

Handling: Data is replicated in HDFS, and tasks running on the failed node are rescheduled on healthy nodes. The JobTracker ensures that progress is maintained.

Failures in MapReduce YARN (Hadoop 2.x and later)

In YARN (Hadoop 2.x and later), failures are handled more efficiently with separate components:

Task Failure: Similar to Classic MapReduce, tasks are retried by the ApplicationMaster, which manages the job lifecycle.

NodeManager Failure: The ResourceManager detects NodeManager failures and reassigns tasks to other nodes.

ApplicationMaster Failure: If the ApplicationMaster fails, the ResourceManager can restart it, ensuring minimal job interruption.

ResourceManager Failure: YARN supports High Availability (HA), so if the ResourceManager fails, a standby can take over without stopping the job.

YARN's architecture provides better fault tolerance, flexibility, and scalability than Classic MapReduce.

Aspect	Classic MapReduce (Hadoop 1.x)	MapReduce YARN (Hadoop 2.x)
Architecture	Single master: **JobTracker** handles everything.	Split roles: **ResourceManager** and **ApplicationMaster** manage resources and jobs.
Fault Tolerance	**JobTracker** is a single point of failure.	**High availability (HA)** for ResourceManager and ApplicationMaster.
Scalability	Limited by **JobTracker** bottlenecks.	Highly scalable with distributed management.
Resource Utilization	Fixed Map and Reduce slots, leading to underutilization.	Dynamic resource allocation via containers for better efficiency.
Task Scheduling	Managed centrally by **JobTracker**.	Handled by **ApplicationMaster** for distributed scheduling.
Framework Support	Only supports **MapReduce** jobs.	Supports multiple frameworks (MapReduce, Spark, etc.).

YARN improves on Classic MapReduce with better **scalability**, **fault tolerance**, and **flexibility**.

Job Scheduling strategies in MapReduce:

FIFO (First In, First Out): Jobs are executed in the order they are submitted.
Pros: Simple and fair in job order.
Cons: Long jobs can block shorter ones.

Capacity Scheduler: Divides resources among different users or groups via queues.
Pros: Fair resource allocation.
Cons: More complex and can lead to inefficiencies.

Fair Scheduler: Allocates resources dynamically to ensure all jobs receive an equal share over time.
Pros: Balances resource use among jobs of varying lengths.
Cons: Overhead from monitoring and adjustments.

Delay Scheduling: Delays task execution to wait for data locality.
Pros: Reduces data transfer overhead.
Cons: May increase job completion time.

Weighted Fair Queueing: Assigns weights to jobs for prioritization.
Pros: Allows prioritization of critical jobs.
Cons: Requires careful weight management.

Load Balancing: Distributes workload evenly across nodes.
Pros: Improves overall utilization
Cons: Can be complex to implement.

Dynamic Job Prioritization: Adjusts job priorities based on urgency and resource use.
Pros: Expedites critical jobs.
Cons: Increases scheduling complexity.

Types of MapReduce Jobs

MapReduce supports various types and formats for processing data. Here's a brief overview of the different types of MapReduce jobs and the input/output formats commonly used:

Types of MapReduce Jobs

Word Count: The classic introductory example where the goal is to count the occurrences of each word in a given text.
Use Case: Text analysis and processing.

Sorting: Sorting data based on keys or values.
Use Case: Organizing data for easier querying or reporting.

Failures in MapReduce YARN (Hadoop 2.x and later)

Join: Combining data from two or more datasets based on a common key.
Use Case: Merging data from different sources, such as databases.

Filtering: Removing unnecessary data based on specific criteria.
Use Case: Data cleaning and preparation.

Aggregation: Summarizing data (e.g., calculating averages, sums).
Use Case: Generating reports or metrics from large datasets.

Machine Learning: Applying algorithms to train models or make predictions.
Use Case: Analyzing large datasets for patterns and insights.

Dr. Rambabu Palaka
Professor
School of Engineering
Malla Reddy University, Hyderabad

Mobile: +91-9652665840
Email: drrambabu@mallareddyuniversity.ac.in